Compiler Construction

Lab 7

Submitted By:

Mahnoor Farrukh (52926)

BSCS-6

Task 1:

(globals)

Task 1.cpp | Task 7.cpp | practice task.cp

```cpp
#include <iostream>
using namespace std;

string input;
int pos = 0;

void S()
{
    if(input[pos] == 'a')
    {
        pos++;
        S();
        if(input[pos] == 'b')
            pos++;
        else
        {
            cout<<"Error\n";
            exit(0);
        }
    }
}

int main()
{
    cout<<"Enter string: ";
    cin>>input;

    S();

    if(pos == input.length())
        cout<<"String Accepted\n";
    else
        cout<<"String Rejected\n";
}
```

```
Enter string: aabb
String Accepted

--------------------------------
Process exited after 6.67 s
Press any key to continue .
```

String rejected:

(globals)

Task 1.cpp | Task 7.cpp | practice ta

```cpp
#include <iostream>
using namespace std;

string input;
int pos = 0;

void S()
{
    if(input[pos] == 'a')
    {
        pos++;
        S();
        if(input[pos] == 'b')
            pos++;
        else
        {
            cout<<"Error\n";
            exit(0);
        }
    }
}

int main()
{
    cout<<"Enter string: ";
```

C:\Users\mahno\OneDrive\Dc

```
Enter string: baba
String Rejected

--------------------------------
Process exited after 3.746 s
Press any key to continue .
```

Recursion Explanation:

Step 1:

Start with:S

Input: aabb

First letter is a, so use:

S - > a S b

Step 2: Now look at the remaining middle part. The remaining input still has another a, so again use:

S - >a S b

Step 3: Now after using two as, no more a is left to process in the middle.

Now it becomes:

a (a S b) b

Input on the left:

bb

No a left, so:

S - > epsilon

Step 4:

a (a epsilon b) b, Since epsilon means nothing, remove it: aabb

Step 5 (Final String):

aabb

Matches input

Task 2:

[*] Untitled1

```cpp
#include <iostream>
#include <string>
using namespace std;

// Input string (e.g., id-id*id)
string input;


int pos = 0;


void E();
void Eprime();
void T();
void Tprime();
void F();


void error()
{
    cout << "Syntax Error\n";
    exit(0);
}

// Function to match a single character
void match(char c)
{
    if (input[pos] == c)
        pos++;              // move forward if matched
    else
        error();            // otherwise error
}

// E → T E'
void E()
{
    T();        // call T
    Eprime();   // then E'
}

// E' → - T E' | ε
void Eprime()
{
    if (input[pos] == '-')   // if '-' found
    {
        match('-');          // match '-'
        T();                 // call T
        Eprime();            // recursive call
    }
    // else epsilon (do nothing)
}

// T → F T'
void T()
{
    F();        // call F
    Tprime();   // then T'
}

// T' → * F T' | ε
void Tprime()
{
    if (input[pos] == '*')   // if '*' found
    {
        match('*');          // match '*'
        F();                 // call F
        Tprime();            // recursive call
    }
    // else epsilon
}

// F → id | (E)
void F()
{
    // Check for "id"
    if (input[pos] == 'i' && input[pos + 1] == 'd')
    {
        pos += 2;   // move forward by 2 characters
    }
    // Check for (E)
    else if (input[pos] == '(')
    {
        match('('); // match '('
        E();        // call E inside brackets
        match(')'); // match ')'
    }
    else
    {
        error();    // invalid case
    }
}

int main()
{
    cout << "Enter expression: ";
    cin >> input;

    E();

    if (pos == input.length())
        cout << "String Accepted\n";
    else
        cout << "String Rejected\n";

    return 0;
}
```

(globals)

Lab7Task2.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

// Input string (e.g., id-id*id)
string input;


int pos = 0;


void E();
void Eprime();
void T();
void Tprime();
void F();


void error()
{
    cout << "Syntax Error\n";
    exit(0);
}

// Function to match a single cha
void match(char c)
{
    if (input[pos] == c)
        pos++;              // ma
    else
        error();            // o
```

C:\Users\mahno\OneDrive\D

```
Enter expression: id-id*id
String Accepted

--------------------------------
Process exited after 30.09 seconds with return value 0
Press any key to continue . . .
```

Task 3:

(globals)

Lab7Task2.cpp | [*] Untitled2

```cpp
#include <iostream>
#include <string>
using namespace std;
string input;


int pos = 0;


void E();
void Eprime();
void T();
void Tprime();
void F();


void error()
{
    cout << "Syntax Error\n";
    exit(0);
}

void match(char c)
{
    if (input[pos] == c)
        pos++;
    else
        error();
}

// E → T E'
void E()
{
    T();
    Eprime();
}

// E' → + T E' | - T E' | ε
void Eprime()
{
    if (input[pos] == '+')
    {
        match('+');
        T();
        Eprime();
    }
    else if (input[pos] == '-')
    {
        match('-');
        T();
        Eprime();
    }
    // else epsilon
}

// T → F T'
void T()
{
    F();
    Tprime();
```

```cpp

// T → F T'
void T()
{
    F();
    Tprime();
}

// T' → * F T' | / F T' | ε
void Tprime()
{
    if (input[pos] == '*')
    {
        match('*');
        F();
        Tprime();
    }
    else if (input[pos] == '/')
    {
        match('/');
        F();
        Tprime();
    }
    // else epsilon
}

// F → id | (E)
void F()
{
    // Check for "id"
    if (input[pos] == 'i' && input[pos + 1] == 'd')
    {
        pos = pos + 2;
    }
    // Check for (E)
    else if (input[pos] == '(')
    {
        match('(');
        E();
        match(')');
    }
    else
    {
        error();
    }
}

int main()
{
    cout << "Enter expression: ";
    cin >> input;

    E();

    if (pos == input.length())
        cout << "String Accepted\n";
    else
        cout << "String Rejected\n";

    return 0;
```

Output string 1:

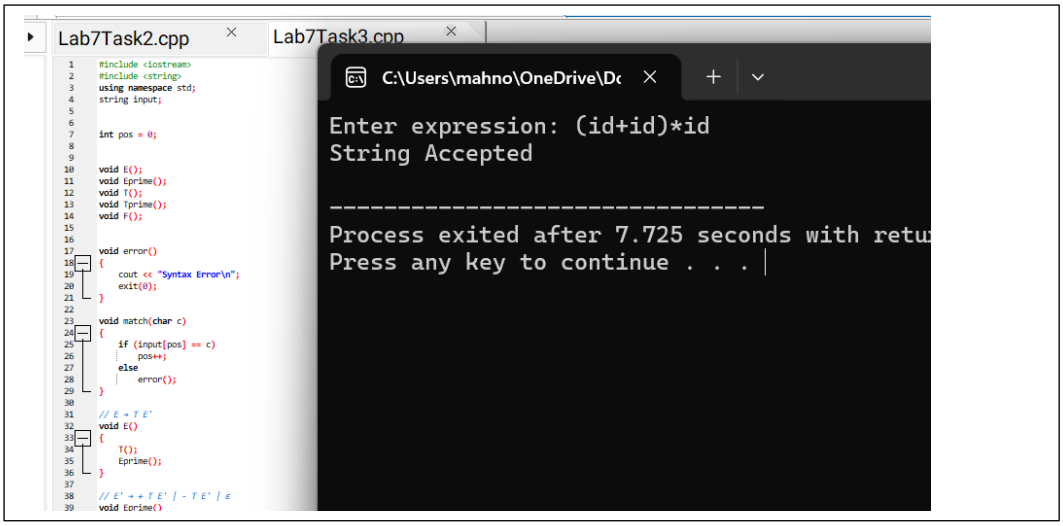
Lab7Task2.cpp | Lab7Task3.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
string input;


int pos = 0;


void E();
void Eprime();
void T();
void Tprime();
void F();


void error()
{
    cout << "Syntax Error\n";
    exit(0);
}

void match(char c)
{
    if (input[pos] == c)
        pos++;
    else
        error();
}

// E → T E'
void E()
{
    T();
    Eprime();
}

// E' → + T E' | - T E' | ε
void Eprime()
{
    if (input[pos] == '+')
    {
        match('+');
        T();
        Eprime();
    }
```

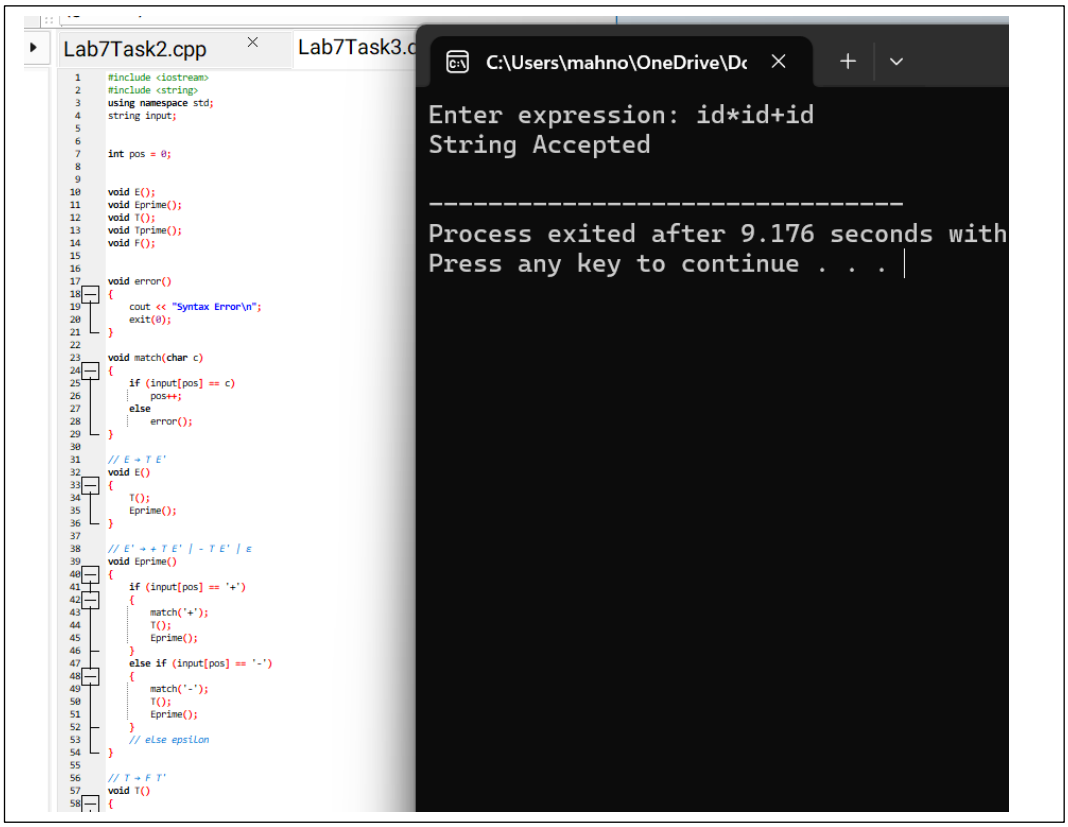
C:\Users\mahno\OneDrive\Dc

```
Enter expression: id+id
String Accepted

--------------------------------
Process exited after 11.87 seconds
Press any key to continue . . .
```

String 2:



string 3:



Explaining Parsing steps:

Start from E

We always start from the start symbol.

E - > T E`
We will first handle T and then E`

Step 2: Go to T

T - > F T`

Here we need F first to solve T

Step 3: Go to F

F ->id
match: id
right input (id) is matched now we move towards left input *id+id

Step 4: Now check T`

T` - > * F T`

Next symbol is * so we use this rule

Match * (id * is matched now we have to match id + id)

Step 5: Again go to F

F - > id

Match id (id*id is matched now we move towards +id)

Step 6: Check T` again

T' -> epsilon

We stop here

Step 7: Now go to E`

E` -> + T E`

As we need + so we will use this rule

Match +

Step 8: Go to T again

T- >F T`

Step 9: Go to F

F - > id
Match id

Here input is matched

All input is used